



Rapport-SAE302-Développer des applications communicantes

Sommaire

Sommaire
Introduction
Réponses au cahiers des charges
Fonctionnalités
Choix des Technologies
Architecture de la Solution
Fonctionnement de l'Architecture
Architecture du Code
Ce qui a été fait et ce qui reste à faire :
Gestion de projet - Diagramme de GANTT
Conclusion

Introduction

Dans ce document de réponses, je vous présente toutes réponses apportées au cahier de charge du projet SAE3.02, qui consiste à concevoir et de développer une architecture multi-serveurs capable de recevoir des requêtes de clients, de compiler et d'exécuter des programmes dans un langage de programmation défini. Les serveurs se repartisse la charge de travail pour une

exécution fluide. L'architecture gère plusieurs clients au même moment et, en cas de surcharge du serveur maître, il délègue les tâches à d'autres serveurs du cluster.

Pour ma part, j'ai choisi de travailler sur un sujet complexe, pour une raison simple qui est que je l'ai mieux appréhendé que le sujet simple, même si je ne suis pas très à l'aise en programmation. Mais j'aimerais toutefois vivre pleinement l'expérience de ce projet, qui me plaît et aller jusqu'au bout. Pour faire avancer ce projet, j'ai utilisé toutes les sources d'information disponibles et discuté avec mes collègues. Je procède au développement de cette application sur Windows en utilisant PyCharm.

Je vous présente dans les lignes qui suivent, les fonctionnalités, les choix des technologies, le fonctionnement de l'architecture, l'architecture du code et ma grande capacité à gérer un projet.

Réponses au cahiers des charges

Fonctionnalités

Le projet demande de créer un cluster avec plusieurs serveurs qui travaillent ensemble. L'idée est que des clients envoient des programmes à un serveur pour qu'il les exécute et renvoie les résultats. Le serveur maître reçoit les programmes que les clients envoient, les envoie aux serveurs esclaves et répartit si la charge est trop lourde. Les serveurs esclaves exécutent les programmes et renvoient les résultats au maître, qui les transmet ensuite aux clients. Le système gère plusieurs clients à la fois, et tout se fait avec des sockets pour permettre la communication. Il est demandé aussi de faire en sorte que le serveur maître surveille la charge pour répartir le travail. En plus, je peux ajouter des options comme sécuriser les échanges, surveiller l'état des serveurs ou démarrer plus de serveurs esclaves automatiquement.

- Fonctionnalités attendues :
 1. Connexion Client-Serveur : Les clients doivent pouvoir se connecter à un serveur via une interface graphique, envoyer des programmes (en Python, Java, C et en C++), et recevoir les résultats d'exécution ou les erreurs de compilation.
 2. Gestion de Plusieurs Clients: Le serveur doit pouvoir gérer plusieurs connexions en même temps et répondre aux demandes de tous les utilisateurs en parallèle.

3. Load Balancing - Répartition de la Charge: Si le serveur principal (ou maître) atteint une certaine limite en termes d'utilisation du CPU ou du nombre de programmes en file d'attente, il doit être capable de déléguer des tâches à un ou plusieurs serveurs (esclaves).
4. Communication : La communication entre le client et le serveur, ainsi qu'entre les serveurs, doit se faire via des sockets.
5. Contraintes Techniques : Les serveurs doivent pouvoir exécuter des programmes dans plusieurs langages et surveiller l'utilisation des ressources (comme le CPU et la mémoire) pour bien gérer la charge.
6. Fonctionnalités Optionnelles : Des améliorations comme le suivi du système, la sauvegarde des données en cas de panne, des mesures de sécurité et la capacité à adapter les serveurs selon les besoins peuvent être ajoutées en plus.

Lien vers le cahier de fonctionnalités :

Document du PROJET - Cahier de fonctionnalités : Conception et Développement d'une Architecture Multi-Serveurs pour Compilation et Exécution de Programmes

En ce qui me concerne, d'abord, j'ai réalisé la connexion client-serveur, où les clients peuvent se connecter au serveur maître et envoyer leurs programmes. Ensuite, j'ai géré la possibilité pour plusieurs clients de se connecter en même temps.

J'ai aussi travaillé sur la répartition des charges. Le serveur maître distribue les tâches aux serveurs esclaves. En cas de surcharge ou de défaillance d'un esclave, le maître redistribue automatiquement la tâche à un autre esclave disponible.

Pour la communication, j'ai utilisé des sockets. ça permet aux clients et serveurs de dialoguer. J'ai aussi pris en compte les erreurs possibles. Concernant l'exécution des programmes, j'ai mis en place un système basé sur le serveur maître et les serveurs esclaves. Le serveur maître reçoit les programmes envoyés par les clients et les répartit aux serveurs esclaves. Les serveurs esclaves, une fois qu'ils reçoivent un programme, le compilent et l'exécutent, puis renvoient le résultat (ou les erreurs) au serveur maître. Le maître, à son tour, transmet ces résultats au client.

Pour la surveillance, j'ai fait en sorte que le serveur maître surveille en permanence la charge du CPU et de la mémoire des deux serveurs esclaves.

Enfin, j'ai intégré des fonctionnalités supplémentaires : Le monitoring. Cette partie affiche l'état des serveurs esclaves, comme leur charge et leur activité en temps réel. ça permet de suivre ce qui se passe dans le cluster et d'identifier problèmes.

Voici les fonctionnalités que j'ai implémentées pour répondre aux objectifs du projet. Je vais les expliquer plus en détail dans les prochaines sections.

Choix des Technologies

Dans cette section, j'explique les choix des technologies que j'ai faits pour le développement du projet.

- Technologies utilisées :
- Python (Modules: OS, SYS)
- Sockets
- Threading
- psutil
- subprocess
- PyQt6
- JSON
- Justification des choix :
- Python est simple à comprendre. Il permet de faire beaucoup de choses rapidement, comme gérer les communications réseau et exécuter des programmes.
- os gère les chemins absolus pour la compilation et l'exécution des programmes.
- sys facilite la gestion des arguments du script et l'arrêt propre des programmes.
- Sockets sont utilisés pour que les différentes parties (client, serveur maître et esclaves) puissent communiquer entre elles. Ils permettent d'envoyer et de recevoir des données facilement sur le réseau.
- Threads permettent de traiter plusieurs demandes en même temps. Par exemple, le serveur maître peut gérer plusieurs clients à la fois sans bloquer les autres.

- psutil sert à surveiller l'état des serveurs esclaves, comme leur utilisation de CPU et de mémoire. Grâce à ces informations, le serveur maître peut décider à quel esclave envoyer une tâche.
- subprocess est utilisé pour exécuter les programmes envoyés par les clients. Cela permet de séparer l'exécution de chaque programme pour éviter que tout le système soit bloqué en cas de problème.
- PyQt6 a été choisi pour créer une interface utilisateur, ça permet aux clients d'envoyer leurs programmes et de recevoir les résultats sans passer par une ligne de commande.
- JSON est utilisé pour stocker les paramètres du système, comme les adresses IP et les ports. C'est un format simple à lire et à modifier, ce qui permet de facilement ajouter de nouveaux serveurs esclaves.

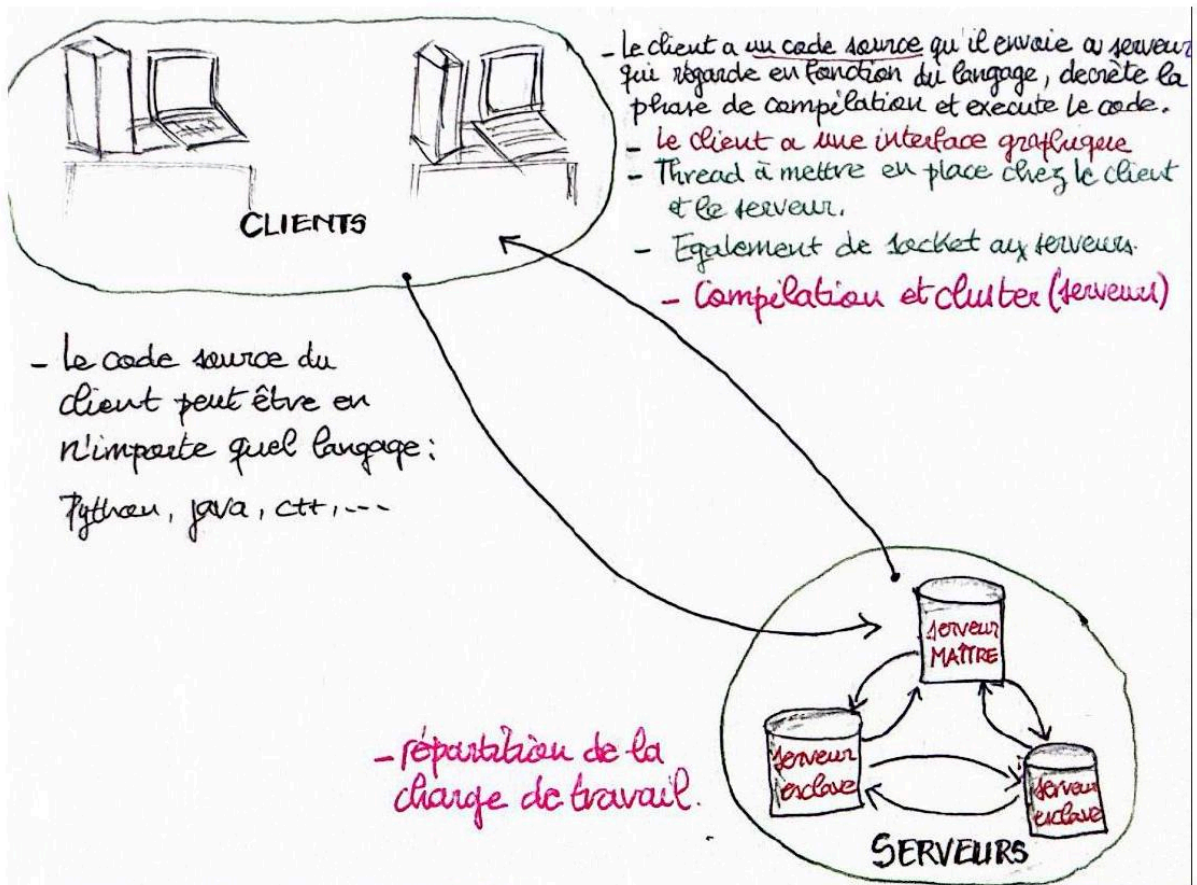
Ces choix ont été faits pour rendre le cluster facile à comprendre et capable de fonctionner correctement même avec plusieurs utilisateurs en même temps.

Architecture de la Solution

Dans cette section, je présente l'architecture du cluster et son fonctionnement global.

- Description de l'architecture :

J'ai mis en place une architecture multi-serveurs organisée autour d'un serveur maître et de plusieurs serveurs esclaves. Cette architecture permet de gérer les programmes envoyés par les clients en répartissant la charge entre les serveurs.



Comment cela fonctionne :

Le serveur maître est le nœud central. Il reçoit les programmes envoyés par les clients via des sockets et les distribue à des serveurs esclaves. Le serveur maître ne traite pas directement les programmes, mais surveille en permanence l'état des serveurs esclaves, par exemple leur utilisation du CPU et de la mémoire, pour répartir intelligemment les tâches. Si un esclave est trop chargé ou indisponible, le maître redirige le programme vers un autre esclave disponible.

Les serveurs esclaves (Esclave 1 et Esclave 2 dans mon implémentation) s'occupent de l'exécution des programmes. Chaque esclave reçoit un programme du maître, l'exécute, et renvoie le résultat ou les erreurs au serveur maître. Ils traitent des langages comme : Python, C, C++, Java et renvoyer des informations sur leur état au maître pour faciliter le monitoring.

Fonctionnement de l'Architecture

Cette section explique en détail comment j'ai mis en œuvre l'architecture pour répondre aux besoins du projet.

Description technique du fonctionnement :

Le serveur maître est développé en Python et utilise des sockets pour communiquer avec les clients et les serveurs esclaves. Quand un client envoie un programme, le serveur maître vérifie l'état des serveurs esclaves (leur CPU et mémoire) à l'aide de la bibliothèque psutil. Ensuite, il choisit un esclave qui peut traiter la tâche et lui envoie le programme. Si tous les esclaves sont occupés, il renvoie un message d'erreur au client. Le maître utilise aussi des threads pour gérer plusieurs clients en même temps, ce qui évite que le système soit bloqué par une seule connexion.

Les serveurs esclaves, également développés en Python, reçoivent les programmes via des sockets, les exécutent ou les compilent selon le langage du code (Python, C, C++, Java). Pour choisir comment l'exécuter : Les programmes Python sont exécutés directement. Les programmes C et C++ sont d'abord compilés avec GCC ou g++ avant d'être exécutés. Les programmes Java sont compilés avec javac, puis exécutés avec java. Pour cela, l'outil subprocess est utilisé pour isoler l'exécution et capturer les sorties ou les erreurs. Chaque esclave renvoie ensuite le résultat ou l'état d'exécution au maître. Ils répondent aussi aux requêtes du maître pour donner des informations sur leur charge CPU et mémoire. En cas de surcharge ou de défaillance d'un esclave, le maître redistribue automatiquement la tâche à un autre esclave disponible.

Du côté client, c'est une application avec une interface graphique créée avec PyQt6. L'utilisateur peut entrer un programme, choisir l'adresse IP et le port du serveur maître, puis envoyer son code d'un simple clic. Une fois le programme envoyé, le client attend la réponse du serveur et affiche les résultats ou les erreurs.

Pour éviter les surcharges, le serveur maître surveille en permanence les ressources des esclaves. Il répartit les programmes de manière intelligente en envoyant les tâches à l'esclave le moins chargé. Une interface graphique sur le maître, développée avec PyQt6, affiche en temps réel l'état des esclaves (charge CPU et mémoire). Ces informations sont mises à jour toutes les 3 secondes.

Le système utilise des sockets pour la communication entre les différentes parties. Les threads permettent de gérer plusieurs clients et esclaves en même temps.

psutil surveille les ressources, et subprocess gère l'exécution et la compilation

des programmes. La configuration du système, comme les adresses IP et les ports, est définie dans un fichier JSON.

Le système est simple. On peut ajouter plus de serveurs esclaves en mettant à jour le fichier de config JSON. Il gère bien la charge grâce au système de surveillance. Les utilisateurs ont une interface simple.

Architecture du Code

Présentation de la structure du code développé et de son organisation.

- Organisation du code :

Les codes du projet sont structurés en trois grandes parties : le serveur maître, les serveurs esclaves, et le client. Chaque partie est organisée de façon simple pour rendre le développement et la gestion plus faciles.

Le serveur maître gère les connexions des clients et délègue les tâches aux serveurs esclaves. Son code est organisé comme suit :

- Initialisation : Le serveur maître lit ses paramètres (adresse IP, port, seuils de charge, liste des esclaves) depuis le fichier de config JSON.
- Gestion des clients: Les connexions des clients sont gérées via des sockets. Chaque client est traité dans un thread séparé pour permettre une gestion parallèle des requêtes.
- Répartition des tâches : Le maître surveille l'état des esclaves (CPU et mémoire) et décide à quel esclave envoyer le programme pour exécution.
- Monitoring : Une interface graphique (PyQt6) affiche l'état des esclaves, avec leurs charges CPU et mémoire.

Les serveurs esclaves exécutent les programmes reçus du maître. Ils sont structurés ainsi :

- Initialisation : Chaque esclave lit ses paramètres (adresse IP, port) depuis le fichier de config JSON.
- Communication: Les esclaves écoutent les messages du maître via des sockets.
- Exécution des programmes : Selon le type de programme (Python, C, C++, Java), l'esclave compile ou exécute le code. Cela se fait avec subprocess pour l'isolation des processus. Les résultats ou les erreurs sont renvoyés au maître.

- Monitoring : Les esclaves répondent aux demandes du maître concernant leur charge CPU et mémoire.

Le client, son code est structuré ainsi :

- Interface utilisateur : L'utilisateur peut saisir ou charger un programme, entrer l'adresse IP et le port du serveur maître, et envoyer le code d'un clic.
- Communication : Le client utilise des sockets pour se connecter au serveur maître, envoyer le programme et recevoir les résultats.
- Gestion des résultats : Les résultats (ou erreurs) sont affichés dans l'interface utilisateur.

Ce qui a été fait et ce qui reste à faire :

J'ai réussi à mettre en place les fonctionnalités principales du projet. Le système permet aux clients de se connecter au serveur maître, d'envoyer des programmes, et de recevoir les résultats. Le serveur maître gère plusieurs clients à la fois et utilise des esclaves pour exécuter les programmes, ce qui assure une bonne répartition des charges. J'ai aussi intégré une interface graphique pour le client, qui simplifie l'utilisation. En termes de fonctionnalités supplémentaires, le monitoring du cluster est opérationnel et affiche l'état des esclaves en temps réel.

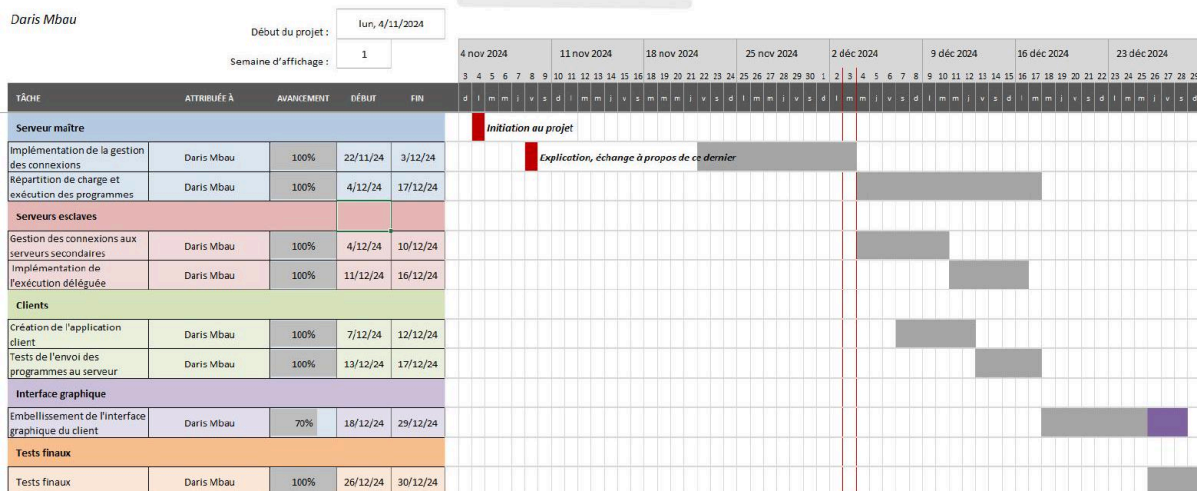
Cependant, certaines choses restent à faire. Par exemple, la persistance des données n'a pas été implémentée, donc les tâches en cours ne sont pas sauvegardées en cas de panne. La sécurité, comme le cryptage ou l'authentification, n'a pas été mise en place non plus. Enfin, des améliorations comme la scalabilité dynamique ou une gestion encore plus robuste des charges pourraient encore être développées.

Gestion de projet - Diagramme de GANTT

Diagramme de GANTT

Dans ce diagramme de GANTT, je montre comment j'organise mon projet. Chaque tâche est bien planifiée avec des dates de début et de fin, et j'ai aussi noté l'avancement en pourcentage. Par exemple, certaines parties comme la gestion des connexions ou les tests des programmes sont terminées à 100%.

Projet - Cluster de calcul



Organisation de mon travail

Pour travailler sur ce projet, je m'organise au mieux dans mon emploi du temps : je travaille dessus dans le train quand je vais en entreprise, le soir, les weekends, et pendant mes temps libres, tant personnels que professionnels. J'ai essayé de trouver un bon équilibre pour avancer sans trop me mettre la pression. Je n'ai pas vraiment compté pas tout le temps que j'ai consacré à ce projet mais je me suis efforcé à finir chaque partie dans le délais que j'ai prévu, et ça m'a motivé de voir que ça progressait bien.

Outils de Gestion de Projet

Pour mener à bien la gestion de mes projets, j'utilise Notion, un outil en ligne pratique qui me permet d'organiser toutes les étapes de mes projets, et tout ce qui y est lié, ça me permet d'avoir accès à mes projets partout où je suis, à tout moment, ainsi je peux avancer dans mes temps libres. Je vous invite à y jeter un œil dans le cadre de ce projet et à découvrir ma grande capacité à gérer un projet : <https://tattered-apparatus-527.notion.site/SAE3-02-D-velopper-des-applications-communicantes-1176da9d6fbf80b5a71ed82d151fd45e?pvs=4>

Conclusion

Ce projet m'a permis de mettre en place une architecture multi-serveurs qui fonctionne bien pour gérer plusieurs clients, exécuter leurs programmes, et répartir les charges entre les serveurs. J'ai appris à gérer les communications réseau, la répartition des tâches, et le développement d'une interface utilisateur simple pour le client. Cependant, j'ai rencontré des difficultés sur certains points.

D'abord, j'ai tenté de mettre en place la sécurité avec le protocole SSL pour sécuriser les échanges entre les clients et le serveur maître. J'ai essayé d'intégrer l'authentification et le cryptage des données, mais j'ai rencontré beaucoup d'erreurs, comme des problèmes de certificats invalides ou des connexions qui échouaient. Je n'ai pas réussi à corriger ces erreurs à temps, et comme il me restait peu de temps pour finaliser le projet, j'ai dû abandonner cette partie. Pourtant, la sécurité est essentielle pour protéger les données échangées et éviter les attaques. Si j'avais eu plus de temps, j'aurais insisté pour la mettre en place.

J'ai aussi tenté d'ajouter la persistance des données, qui consiste à sauvegarder les tâches en cours pour qu'elles puissent être reprises en cas de panne d'un serveur. ça aurait permis d'améliorer la robustesse du système, mais je n'ai pas réussi à la faire fonctionner. C'était difficile de gérer la récupération des données et leur transfert automatique vers d'autres serveurs.

Enfin, la scalabilité, qui permet d'ajouter ou de retirer des serveurs selon la charge, pourrait encore être améliorée. Avec plus de temps, je pourrais développer cette fonctionnalité pour rendre le système encore plus performant.

Malgré ça, ce projet m'a beaucoup appris. par exemple comment gérer plusieurs clients en même temps, répartir les tâches entre les serveurs, J'ai aussi appris à résoudre des problèmes, même si je n'ai pas tout réussi.

Les limites du projet sont donc au niveau de la sécurité, de la persistance de données, et de l'évolutivité. Mais ce projet fonctionne bien pour ce qui a été fait, comme exécuter des programmes, gérer les clients, et afficher l'état des serveurs.

Pour aller plus loin, je pourrais ajouter une vraie sécurité, améliorer la gestion des serveurs, et intégrer des sauvegardes automatiques des tâches. Ce projet peut aussi être élargi pour inclure plus de types de programmes ou permettre à plus de serveurs de travailler ensemble.

Je trouve que ce projet est formateur. Il m'a permis de mieux comprendre comment les systèmes travaillent ensemble et de voir ce qui est important dans un projet complexe. C'est une bonne base pour apprendre et m'améliorer dans des projets futurs.